PYTHON 101 /
BLOCK 25

Creating and Calling Functions

Day 4 Morning - Functions and Program Design

30 min teaching + 30 min exercises

```
def calculate_percentage(completed, total):
    percentage = completed / total * 100
    return percentage
```

```
result = calculate_percentage(18, 24)
print(f"{result:.1f}%")
```

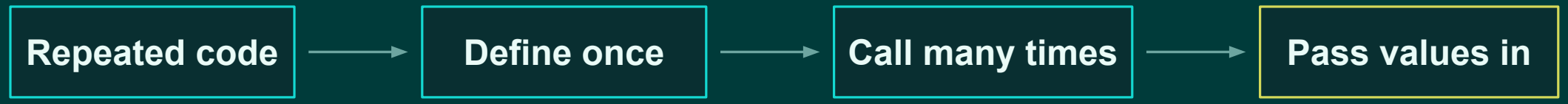
Goal: move from repeated script logic to reusable named steps.

BLOCK 25

Where this block fits

By the end, students can define functions, call them, and pass values into them.

Instruction flow



Practice flow

30 minutes instruction
- Mental model
- Function anatomy
- Parameters and arguments
- Execution flow

30 minutes exercises
- Convert old calculations
- Return numbers, booleans, and strings
- Test each function independently

BLOCK 25

Why functions exist

When the same idea appears in several places, change becomes risky.

```
completed_tasks = 18
total_tasks = 24

percentage = completed_tasks /
total_tasks * 100

print(f"{percentage:.1f}%")
```

```
completed_tasks = 42
total_tasks = 50

percentage = completed_tasks /
total_tasks * 100

print(f"{percentage:.1f}%")
```

```
completed_tasks = 87
total_tasks = 100

percentage = completed_tasks /
total_tasks * 100

print(f"{percentage:.1f}%")
```

The calculation is copied three times.
If the rule changes, every copy must be found and changed correctly.

BLOCK 25

A function names a reusable step

Define the logic once. Then call it wherever it is needed.

```
def calculate_percentage(completed, total):
    percentage = completed / total * 100
    return percentage
```

→

```
result_1 = calculate_percentage(18, 24)
result_2 = calculate_percentage(42, 50)
result_3 = calculate_percentage(87, 100)
```

Mental model
A function is a named recipe.
The recipe sits there until you ask Python to use it.

BLOCK 25

Anatomy of a function

Students should be able to point to each part of the definition.

```
def calculate_percentage(completed, total):
    percentage = completed / total * 100
    return percentage
```

def
Defines a function

name
calculate_percentage

parameters
completed, total

colon
starts the body

indented body
code inside function

return
sends value back

The indentation is not decoration. It tells Python what belongs to the function.

BLOCK 25

Definition is not execution

Creating a function does not automatically run it.

```
def say_hello():
    print("Hello")
```

Nothing prints yet.
Python has only learned the recipe.

```
say_hello()
```

Now the function runs.
This is a function call.

Definition = describe what the function does

Call = actually execute the function

BLOCK 25

Functions can have no parameters

Some functions do the same thing every time.

```
def print_header():
    print("Record Processing Report")
    print("------------------------")

print_header()
```

When useful

No-parameter functions are good for repeated fixed actions, such as printing a heading or showing a menu.

Caution

Most useful functions need information from the caller. That information comes through parameters.

BLOCK 25

Functions can accept one input

A parameter is a local name for a value supplied by the caller.

```
def double_value(value):
    result = value * 2
    return result

answer = double_value(5)

print(answer)
```

Inside the function, `value` is the name Python uses for the argument.

BLOCK 25

Functions can accept multiple inputs

Multiple parameters let a function combine values from the caller.

```
def calculate_total(price, quantity):
    return price * quantity

total = calculate_total(12.50, 8)
```

price receives 12.50

quantity receives 8

return value is 100.0

The function does not care where the values came from. It only needs the inputs it was promised.

BLOCK 25

Parameters versus arguments

The same idea from two different sides of the function call.

```
def calculate_total(price, quantity):
    return price * quantity
```

```
total = calculate_total(12.50, 8)
```

Parameters
Names written in the function definition.
They describe what information the function expects.

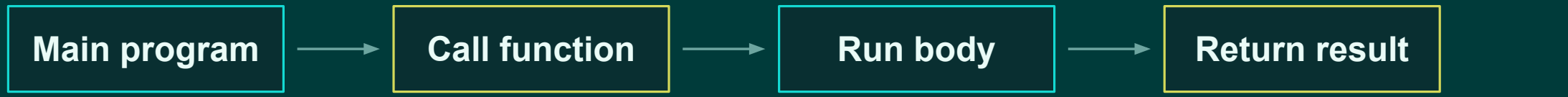
Arguments
Actual values supplied when the function is called.
They fill the parameter names.

Memory hook: parameters are placeholders; arguments are the real values.

BLOCK 25

Execution temporarily enters the function

After return, Python continues where the call happened.




```
print("Starting")

def double_value(value):
    result = value * 2
    return result

answer = double_value(5)

print(answer)
print("Finished")
```

Continue main program

Output order:
Starting
10
Finished

BLOCK 25

Returning a value to the caller

`return` sends a result back to the place where the function was called.

```
def megabytes_to_gigabytes(megabytes):
    gigabytes = megabytes / 1000
    return gigabytes

result = megabytes_to_gigabytes(2500)

print(result)
```

The returned value can be stored in a variable, printed, compared, or passed into another function.

For this block: return means “give the answer back to the calling code.”

BLOCK 25

Function body and indentation

Everything indented under `def` belongs to the function.

```
def show_status(status):
    print("Status Report")
    print(status)

print("Outside the function")
```

Inside
The two indented print statements run only when show_status() is called.

Outside
The final print statement is not part of the function because it is not indented.

In Python, indentation is structure.

BLOCK 25

Common beginner mistakes

These are normal errors. The goal is to recognize them quickly.

```
def say_hello():
    print("Hello")

# Forgot to call say_hello()
```

Defined but never called

```
def calculate_total(price, quantity)
    return price * quantity
```

Missing colon after header

```
def calculate_total(price, quantity):
    total = price * quantity

result = calculate_total(10, 5)
```

No return, so result is None

Debugging question: did I define it, call it, and return the value I need?

BLOCK 25

Instructor mini-demo

Build one function live, then call it with several values.

1. Start with repeated percentage code.
2. Move the repeated calculation into a function.
3. Call the function three times with different data.
4. Change the calculation in one place and rerun.

```
def percent(done, total):
    return done / total * 100

result = percent(18, 24)
print(f"{result:.1f}%")

result = percent(42, 50)
print(f"{result:.1f}%")
```

Live emphasis: functions let us change behavior in one place.

BLOCK 25

Guided exercise setup

Students convert earlier calculations into small functions.

Exercise rule
Write the function first.
Then call it with several test values.
Then print the returned result.

Starter checklist
- Use def
- Use meaningful names
- Include parameters when values change
- Return the result
- Call the function

Timing: 30 minutes total practice

BLOCK 25

Exercise 1: megabytes to gigabytes

Convert a familiar calculation into a reusable function.

```
def megabytes_to_gigabytes(megabytes):
    gigabytes = megabytes / 1000
    return gigabytes

result = megabytes_to_gigabytes(2500)

print(result)
```

Task
Run the example, then test at least three more values: 1000, 1500, and 5000.

Time box
5 minutes

Extension: should this use 1000 or 1024? Explain the choice.

BLOCK 25

Exercise 2: completion percentage

Create a function that accepts two changing values.

```
def calculate_completion(completed, total):
    return completed / total * 100

print(calculate_completion(18, 24))
print(calculate_completion(42, 50))
print(calculate_completion(87, 100))
```

Task
Create the function. Then test 0%, 50%, 75%, and 100% cases.

Discuss
What happens if total is 0?
Do not fix it yet; just observe.

This function is a foundation for later validation and testing.

BLOCK 25

Exercise 3: return a Boolean

A function can return True or False.

```
def is_temperature_valid(temperature):
    return 60 <= temperature <= 80

print(is_temperature_valid(72))
print(is_temperature_valid(45))
print(is_temperature_valid(90))
```

Task
Create the function and test values below, inside, and above the valid range.

Mental model
A Boolean function answers a yes/no question.

Naming hint: Boolean functions often start with is_, has_, or can_.

BLOCK 25

Exercise 4: normalize a status

Functions are useful for cleaning repeated text patterns.

```
def normalize_status(status):
    return status.strip().lower()

print(normalize_status(" ACTIVE "))
print(normalize_status("Inactive"))
print(normalize_status(" retired "))
```

Task
Test several inputs with different capitalization and whitespace.

Notice
The caller does not need to remember the cleanup details every time.

This is the first step toward a reusable data-cleaning pipeline.

BLOCK 25

Exercise 5: validate a file type

Combine cleanup with a Boolean result.

```
def is_supported_file(file_name):
    file_name = file_name.strip().lower()

    return (
        file_name.endswith(".csv")
        or file_name.endswith(".json")
    )
```

Task
Test these cases:
records.csv
records.json
records.txt
REPORT.CSV

Time box
6 minutes

Key idea: normalize first, then validate the normalized value.

BLOCK 25

Applied challenge: find reusable operations

Turn Day 2 and Day 3 patterns into named functions.

Create at least four functions
normalize_record_id(...)
calculate_percentage(...)
is_supported_file(...)
is_valid_record_count(...)

For each function
1. Define it
2. Call it with normal values
3. Call it with edge values
4. Print the returned result

BLOCK 25

Block 25 wrap-up

Students should leave with a beginner mental model for functions.

Students can explain
- What a function is
- Definition versus call
- Parameters versus arguments
- Function body and indentation
- How return sends a result back

Students can do
- Convert repeated calculations
- Write no-parameter functions
- Write functions with inputs
- Return numbers, strings, and booleans
- Test functions independently

Next block: return, multiple inputs, and predictable function behavior.